



《AI大模型Ragent项目》——深度解析知识库定时同步的架构设计

来自： 拿个offer-开源&项目实战

 马丁

2026年03月31日 22:44

前一篇文章把 startChunk 的异步处理链路讲完了：用户点击执行分块，系统用 RocketMQ 事务消息把状态更新和消息投递绑在一起，然后由消费者异步执行分块。文章结尾提到，startChunk 方法中调用了 scheduleService.upsertSchedule(documentD0) 注册定时任务，但底层调度机制还没讲。

这一篇来讲：系统怎么按照用户配置的 scheduleCron 表达式，定时抓取远程 URL 文档，检测有没有变化，有变化才重新分块。因为内容比较多，我们拆成上下两篇：

上篇（本篇）：讲数据模型、架构设计、分布式锁、变更检测，让你理解这套系统是怎么设计的

下篇：讲调度引擎、执行流程、异常恢复，让你理解这套系统是怎么跑起来的

为什么需要定时同步

先说业务场景。

企业的文档通常放在飞书、Notion、钉钉这类在线协作平台上。这些文档会不断更新：产品经理改需求文档，HR 更新员工手册，运维修正操作规范。但文档作者不会每次改完都记得去知识库重新上传一遍。

如果没有定时同步机制，会出现两个问题：

用户问的是最新制度，知识库回答的却是上周版本

文档作者已经在线上修正了错误，知识库还在用旧内容继续回答

本地文件上传适合一次性导入。文件传完之后，知识库里的内容和对象存储里的文件是一一对应的，除非用户重新上传，否则内容不会变。但 URL 来源不是这样。URL 对应的是一个远程资源，它背后可能是一份不断更新的在线文档。今天的内容和明天的内容，URL 不变，文件内容却可能已经变了。

所以 URL 来源的文档支持两个额外配置：

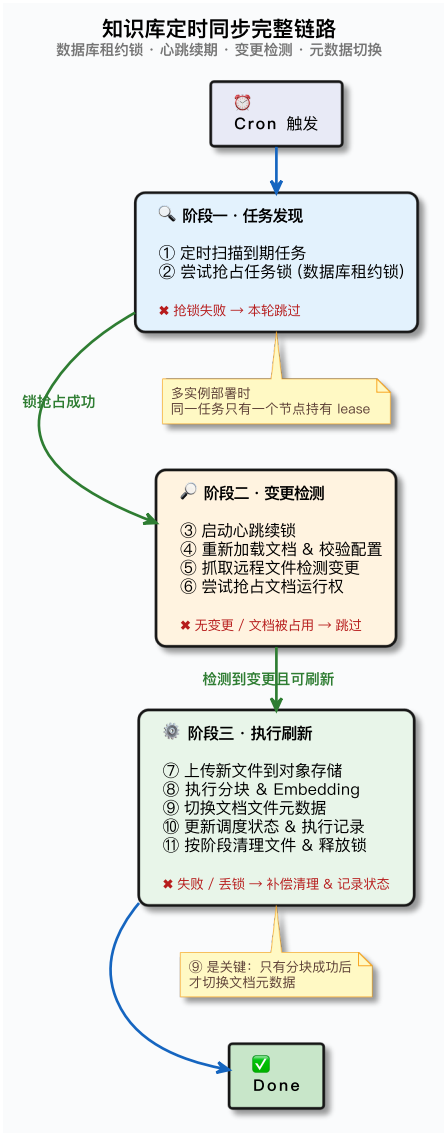
scheduleEnabled：是否开启定时同步

scheduleCron：同步周期（cron 表达式）

当这两个参数配置好之后，系统会自动完成这条链路：定时扫描 → 抢任务锁 → 拉远程文件 → 检测变更 → 抢文档运行权 → 重新分块 → 更新状态。

完整流程概览

先用一张时序图建立整体认知。这张图展示的是一次成功的定时同步完整链路，省略了失败分支和边界情况，让你先看清楚主线：



从这张图能看出来，定时同步不是一个简单的 @Scheduled 方法，而是一套完整的调度系统。它至少要解决四个问题：

1. 怎么扫描到期任务
2. 多实例部署时，怎么防止同一个任务被重复执行
3. 怎么判断远程文档有没有变
4. 分块过程中如果失败、丢锁、切换文件失败，怎么做补偿

要理解这套机制，最好先从数据模型入手。因为所有的调度逻辑、锁机制、状态管理，最终都要落到数据库表上。

定时任务表设计

在讲具体的调度逻辑之前，先把数据模型理清楚。定时同步机制依赖两张表：任务主表和执行历史表。

1. 设计思路

先想清楚这两张表要解决什么问题。

任务主表要回答的问题：

- 这个文档的定时任务是什么配置？（cron 表达式、是否启用）
- 下次什么时候该执行？（nextRunTime）
- 上次执行得怎么样？（lastStatus、lastError）
- 上次远程文件是什么状态？（lastEtag、lastModified、lastContentHash）
- 现在有没有实例正在执行这个任务？（lockOwner、lockUntil）

执行历史表要回答的问题：

- 这个任务历史上执行了多少次？
- 每次执行的结果是什么？（SUCCESS / FAILED / SKIPPED）

每次执行花了多长时间？（startTime、endTime）

每次执行时远程文件的状态是什么？（etag、lastModified、contentHash）

主表只保留当前状态，历史表保留完整流水。这样设计的好处是：

主表查询快，只需要关心现在该不该运行

历史表可以追溯，方便排查问题和统计分析

2. 任务主表：t_knowledge_document_schedule

先看表结构，再逐组解释字段含义：

```
CREATE TABLE t_knowledge_document_schedule (
  id          VARCHAR(20) NOT NULL PRIMARY KEY,
  doc_id      VARCHAR(20) NOT NULL,
  kb_id       VARCHAR(20) NOT NULL,
  cron_expr   VARCHAR(64),
  enabled     SMALLINT DEFAULT 0,
  next_run_time  TIMESTAMP,
  last_run_time  TIMESTAMP,
  last_success_time TIMESTAMP,
  last_status   VARCHAR(16),
  last_error    VARCHAR(512),
  last_etag     VARCHAR(256),
  last_modified VARCHAR(256),
  last_content_hash VARCHAR(128),
  lock_owner   VARCHAR(128),
  lock_until   TIMESTAMP,
  CONSTRAINT uk_doc_id UNIQUE (doc_id)
);
```

注意 doc_id 上有唯一约束，这意味着一份文档最多只有一条 schedule 记录。

字段可以分成四组理解：

基础信息（描述这个任务是谁、多久跑一次、是否启用）：

- docId：关联的文档 ID
- kbId：知识库 ID
- cronExpr：cron 表达式，定义同步周期（比如 0 0 * * * 表示每天零点）
- enabled：是否启用，0 禁用 / 1 启用

调度状态（决定什么时候该跑，以及上次跑得怎么样）：

- nextRunTime：下次执行时间，扫描器根据这个字段找到期任务
- lastRunTime：上次执行时间
- lastSuccessTime：上次成功时间
- lastStatus：上次执行结果（SUCCESS / FAILED / SKIPPED）
- lastError：上次失败原因

这组字段的设计逻辑是：扫描器每次查询 nextRunTime <= now 的记录，执行完后更新 lastRunTime 和 nextRunTime。

变更检测（存上一次远程文件的检测结果，给下一次比对用）：

- lastEtag：上次远程文件的 ETag（HTTP Header）
- lastModified：上次远程文件的 Last-Modified（HTTP Header）
- lastContentHash：上次远程文件的 SHA-256 哈希

这三个字段是变更检测的关键。下次定时触发时，系统会优先比对远程文件返回的 ETag / Last-Modified 和这里存下来的历史值；如果第一层没有命中，再退化到内容哈希比对。

分布式锁（支撑数据库租约锁机制）：

- lockOwner：谁持有锁（lockToken）
- lockUntil：锁什么时候过期

这两个字段实现了一个简单但对当前场景来说比较合适的分布式锁。多个实例同时扫描到同一个任务时，只有一个能成功把 lockOwner 改成自己的 token。

3. 执行历史表：t_knowledge_document_schedule_exec

主表只保留最近一次的结果，详细历史要放在 exec 表里：

```
CREATE TABLE t_knowledge_document_schedule_exec (
  id          VARCHAR(20) NOT NULL PRIMARY KEY,
  schedule_id VARCHAR(20) NOT NULL,
  doc_id      VARCHAR(20) NOT NULL,
  kb_id       VARCHAR(20) NOT NULL,
  status      VARCHAR(16) NOT NULL,
  message     VARCHAR(512),
  start_time  TIMESTAMP,
  end_time    TIMESTAMP,
  file_name   VARCHAR(512),
  file_size   BIGINT,
  content_hash VARCHAR(128),
  etag        VARCHAR(256),
  last_modified VARCHAR(256)
);
```

这张表保存的是每次执行的流水。它和主表的区别：

- 主表回答这个任务现在是什么状态
- exec 表回答它历史上每次执行发生了什么

为什么需要执行历史表？

假设你配置了每小时同步一次，一天就是 24 次执行。如果只在主表保留最近一次的结果，就无法回答这些问题：

- 这个任务过去一周执行了多少次？
- 成功率是多少？
- 哪些时间段经常失败？
- 失败原因是什么？

exec 表保留完整的执行历史，方便排查问题和统计分析。

4. 表结构和业务逻辑的对应关系

理解了表结构，再看后面的代码会容易很多。这里先建立一个对应关系：

业务逻辑	对应的表字段
扫描到期任务	enabled=1 AND nextRunTime <= now AND lockUntil < now
抢锁	更新 lockOwner 和 lockUntil, 条件是 lockUntil < now
续锁	更新 lockUntil, 条件是 lockOwner 匹配
释放锁	把 lockOwner 和 lockUntil 设为 NULL
变更检测	比对远程文件的 ETag / Last-Modified / SHA-256 和 lastEtag / lastModified / lastContentHash
记录执行结果	插入 exec 记录, 更新主表的 lastStatus / lastRunTime / nextRunTime

有了这个对应关系，后面看代码时就能快速理解“这段代码在操作哪些字段，为什么要这样操作”。

架构设计：职责分离

理解了数据模型，再来看代码架构。

1. 为什么要拆分职责

如果把定时同步的所有逻辑都塞进一个类里，会出现几个问题：

单一类职责过重：一个方法几百行，既要扫描任务，又要抢锁，又要下载文件，又要分块，又要更新状态。改一处可能影响其他逻辑，难以理解和测试。

无法单独测试某个环节：比如只想测试变更检测逻辑，但它和锁管理、状态更新混在一起，没法单独测试。

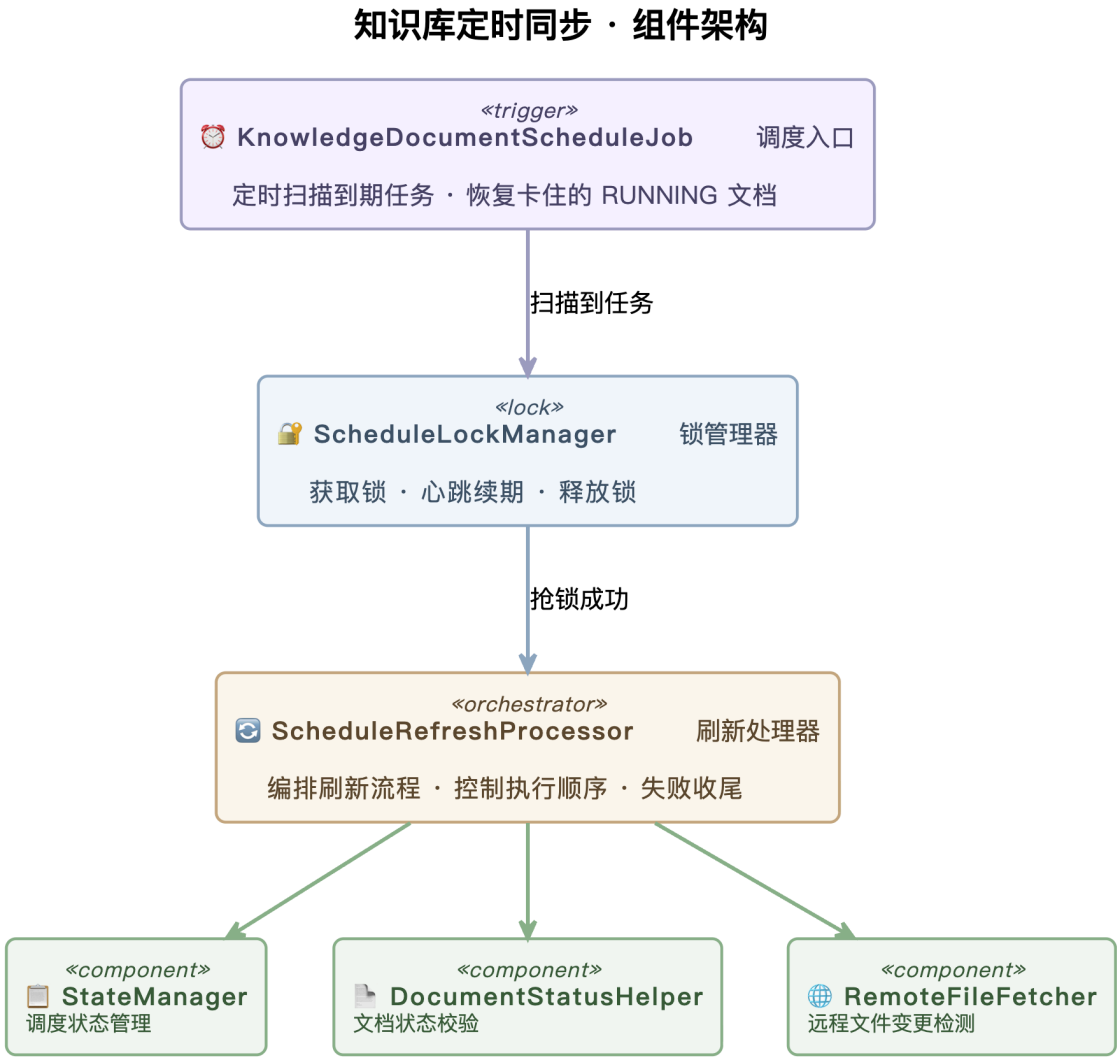
难以扩展：比如想把数据库锁换成 Redis 锁，或者想换一种变更检测策略，都需要改动很多地方。

所以系统把定时同步拆分成五个核心组件，每个组件只负责一件事。这样做的好处是：

- 每个组件职责清晰，容易理解
- 可以单独测试每个组件
- 需要修改某个环节时，只需要改对应的组件

2. 五个核心组件

先看一张组件协作图，建立整体认知：



接下来逐个介绍这五个组件。

2.1 KnowledgeDocumentScheduleJob：调度入口

这是整个定时同步的入口，负责两件事。

第一件事：扫描到期任务

每隔一段时间（默认 10 秒）执行一次，查询满足条件的任务：

```
enabled = 1: 任务处于启用状态
nextRunTime <= now: 已经到执行时间了
lockUntil < now: 当前没有其他实例持有锁
```

找到任务后，不是直接执行，而是先尝试获取锁。抢锁成功的实例才能执行，抢锁失败的实例直接跳过。

第二件事：恢复卡住的文档

每分钟扫描一次，把卡在 RUNNING 状态超过阈值（默认 30 分钟）的文档重置为 FAILED。这是进程崩溃后的兜底恢复机制。

这个组件的设计思路是：负责找到该跑的任务和兜底恢复异常状态，不负责具体的刷新逻辑。

2.2 ScheduleLockManager：锁管理器

负责分布式锁的获取、续期、释放，以及自动心跳续锁机制。

这个组件解决的核心问题是：多个实例同时扫描到同一个任务时，怎么保证只有一个实例在执行。

它提供了四个核心方法：

```
tryAcquire(scheduleId, now): 尝试获取锁，返回 lease 或 null

renew(lease): 续期锁，返回是否成功

release(lease): 释放锁

startHeartbeat(lease): 启动自动心跳续锁
```

其中最特色的是自动心跳续锁机制。传统的做法是业务代码在每个耗时操作前手动调用 `renew(lease)`，但这样容易遗漏。自动心跳机制启动一个后台线程，周期性自动续锁，业务代码只需在关键阶段检测锁是否失效即可。

这个组件的设计思路是：把锁的复杂性封装起来，业务代码不需要关心锁的续期细节。

2.3 ScheduleRefreshProcessor：刷新处理器

负责完整的刷新流程编排，是定时同步的总控台。

它不负责单独完成某一件事，而是决定：这些动作应该按什么顺序发生，中间哪一步要跳过，哪一步失败后该怎么收尾。

核心方法 `process(ScheduleLockLease lease)` 管理整个刷新流程：

1. 启动心跳
2. 校验任务合法性
3. 创建执行记录
4. 变更检测
5. 抢占文档运行权
6. 上传新文件
7. 执行分块
8. 应用文件元数据
9. 更新状态
10. 文件清理
11. 关闭心跳
12. 释放锁

它还引入了 `Phase` 枚举追踪执行阶段（INIT → DOC_OCCUPIED → CHUNK_STARTED → CHUNK_COMPLETED → FILE_SWITCHED），在关键阶段检测锁是否失效，根据 `Phase` 精细化控制文件清理策略。

这个组件的设计思路是：把流程编排和具体实现分离，让主线逻辑清晰可见。

2.4 ScheduleStateManager：状态管理器

负责更新 `schedule` 表和 `exec` 表的状态。

这个组件解决的核心问题是：更新状态时必须感知锁所有权，防止覆盖其他实例的状态。

它提供的核心方法都采用 `xxxIfOwned` 模式：

```
markSuccessIfOwned(lease, ctx, fetchResult, stored): 成功时更新状态

markFailedIfOwned(lease, ctx, error): 失败时更新状态

markSkippedIfOwned(lease, ctx, fetchResult): 跳过时更新状态

disableIfOwned(lease, reason): 禁用任务
```

这些方法更新 `schedule` 主表时都会检查 `lockOwner` 是否匹配当前 `lockToken`。如果不匹配（锁已失效），`schedule` 主表更新失败，但仍会更新 `exec` 记录，并在 `message` 中附加（调度锁已失效，未写回调度状态）标记。

这个组件的设计思路是：状态更新必须是锁感知的，不能盲目覆盖其他实例的状态。

2.5 DocumentStatusHelper：文档状态助手

负责文档状态的 CAS 更新和恢复。

这个组件解决的核心问题是：防止定时刷新和手动分块同时跑同一份文档。

它提供了四个核心方法：

```
tryMarkRunning(docId): CAS 抢占文档运行权，条件 ne(status, RUNNING)

markFailedIfRunning(docId): 失败时恢复文档状态

applyRefreshedFileMetadata(docId, stored): 成功后应用新文件元数据

recoverStuckRunning(timeoutMinutes): 恢复卡住的 RUNNING 文档
```

这个组件提供了第二层并发控制。第一层是任务级别的数据库租约锁（防止同一个 schedule 被多个实例同时执行），第二层是文档级别的 CAS 抢占（防止定时刷新和手动分块同时跑同一份文档）。
这个组件的设计思路是：用 CAS 操作确保文档状态的原子性更新。

3. 组件协作关系

这五个组件的协作关系可以概括成一句话：

ScheduleJob 扫描任务 → LockManager 抢锁 → RefreshProcessor 编排流程 → StateManager 更新状态 + DocumentStatusHelper 管理文档状态 + RemoteFileFetcher 检测变更

每个组件只负责自己的职责，通过方法调用协作完成整个定时同步流程。

分布式锁设计

分布式锁是本篇的核心设计之一。前面提到调度引擎会尝试获取锁，现在来详细讲讲这个锁是怎么设计的。

1. 为什么需要分布式锁

先看一个具体场景。
假设你的应用部署了 3 个实例（实例 A、实例 B、实例 C），每个实例都运行着定时扫描任务。某个 schedule 的 nextRunTime 是 2026-03-30 08:00:00，到了这个时间点，3 个实例几乎同时执行扫描。
没有锁的情况下会发生什么？

- 1. 实例 A 扫描到这条记录，认为这个任务该我来跑
- 2. 实例 B 也扫描到这条记录，也认为这个任务该我来跑
- 3. 实例 C 也扫描到这条记录，也认为这个任务该我来跑
- 4. 3 个实例同时调用 scheduleRefreshProcessor.process()
- 5. 同一个远程文件被下载 3 次
- 6. 同一份文档被分块 3 次
- 7. 3 个实例都在写 schedule 主状态，互相覆盖

这就是典型的分布式环境下的并发冲突。

有锁的情况下会发生什么？

- 1. 实例 A 扫描到这条记录，尝试获取锁，成功
- 2. 实例 B 也扫描到这条记录，尝试获取锁，失败（因为实例 A 已经持有锁）
- 3. 实例 C 也扫描到这条记录，尝试获取锁，失败（因为实例 A 已经持有锁）
- 4. 只有实例 A 执行刷新逻辑，实例 B 和 C 直接跳过

分布式锁的作用：同一个 schedule 在同一时刻，只能有一个实例在执行。

2. 为什么选择数据库锁

常见的分布式锁方案有两种：Redis 分布式锁和数据库锁。为什么这里选择数据库锁？
先看两种方案的对比：

维度	Redis 分布式锁	数据库租约锁
性能	高，Redis 是内存操作	相对较低，数据库是磁盘操作
通用性	通用，可以用于各种场景	只适合和业务数据在同一个库的场景
成熟度	成熟，有 Redisson 等现成方案	需要自己实现
依赖	需要额外维护 Redis 高可用	不新增基础设施
数据一致性	锁状态和业务数据分离	锁和任务状态在一张表里
异常排查	需要跨系统查询（查数据库 + 查 Redis）	只需要查数据库

对这个场景来说，有几个特点：

调度状态本来就存在数据库里：t_knowledge_document_schedule 表已经存在了，再加两个字段（lockOwner、lockUntil）就能把锁也放进去。

锁的粒度是 schedule 级别：锁和表的主键对应，一条 schedule 记录对应一把锁。

定时任务的扫描频率不高：默认 10 秒一次，每次最多 20 个任务，数据库完全扛得住。

出问题时排查更方便：运维可以直接查数据库看锁状态（哪个实例持有锁、锁什么时候过期），不需要再去查 Redis。

所以这里选择数据库锁，架构上更收敛。对于定时任务调度这个场景，数据库锁够用了。

3. 为什么这里叫租约锁

这里其实不是租约锁和分布式锁二选一。

更准确地说，当前实现本身就是一种分布式锁，只不过它采用的是**租约型**的持锁方式。

分布式锁强调的是作用范围：多个实例之间要互斥

租约锁强调的是持锁模型：锁不是一直持有，而是带有租期，到期会自动失效，持有者还能续租

所以，租约锁是另一种形式的分布式锁，一把带租期的分布式锁。对比着理解，是一个 Redisson 看门狗简单实现。

3.1 当前项目里的租约体现在哪

放到当前实现里，租约语义非常具体：

lockOwner：谁持有这把锁

lockUntil：这把锁什么时候到期

抢锁时，当前实例不是去等待别人释放，而是直接尝试更新：

如果 lockUntil 为空，说明当前没人持有锁

如果 lockUntil < now，说明旧锁已经过期

只有这两种情况下，当前实例才能把 lockOwner 改成自己的 token

这就是租约的核心意思：

锁不是永久占有

锁只在一段时间内有效

到期不续租，别人就可以接管

而心跳续锁做的事，本质上就是不断把 lockUntil 往后推，相当于给这把锁续租。

3.2 为什么它适合定时调度

定时任务最怕两件事：

1. 多个实例同时执行同一个任务
2. 某个实例挂了以后，锁一直不释放

租约型分布式锁正好同时解决这两个问题：

抢锁成功的实例继续执行

其他实例这轮直接跳过

如果持锁实例挂了，锁会在租期结束后自动失效

如果任务执行时间比较长，还可以通过心跳续租

所以在这个场景里，更准确的名字应该是：**数据库实现的租约型分布式锁**。它之所以叫租约锁，是因为它的语义更像租房：

现在这段时间归你使用

到期不续租，别人就能接手

如果你还在用，就主动续租

4. 锁的获取、续期和释放

理解了租约锁的概念，再来看具体实现。

4.1 获取锁

获取锁的本质是：尝试把 lockOwner 改成自己的 token，只有当锁不存在或已过期时才能改成功。

先看 ScheduleLockManager.tryAcquire() 方法要做什么：

1. 生成一个唯一的 lockToken（标识这次锁的持有者）

- 2. 计算锁的过期时间 lockUntil（当前时间 + 锁持有时间）
- 3. 尝试更新数据库，把 lockOwner 改成自己的 token
- 4. 更新成功返回 lease，更新失败返回 null

核心是第 3 步的更新条件：

```
UPDATE t_knowledge_document_schedule
SET lockOwner = ?, lockUntil = ?
WHERE id = ?
AND (lockUntil IS NULL OR lockUntil < now)
```

这条 SQL 的语义是：只有当锁不存在（lockUntil IS NULL）或锁已过期（lockUntil < now）时，才允许更新。这就是典型的 CAS（Compare-And-Swap）操作。多个实例同时执行这条 SQL，只有一个能更新成功，其他的都会返回 affected rows = 0。

为什么要生成唯一的 lockToken？

因为后面续锁和释放锁时，需要验证这把锁是不是我持有的。如果不验证，可能会出现这种情况：

- 1. 实例 A 获取锁，开始执行任务
- 2. 实例 A 执行很慢，锁过期了
- 3. 实例 B 获取锁，开始执行同一个任务
- 4. 实例 A 执行完了，释放锁
- 5. 实例 A 误把实例 B 的锁释放了

所以每次获取锁时都生成一个唯一的 token，续锁和释放锁时都要验证 token 是否匹配。

锁的过期时间怎么设置？

默认配置是 900 秒（15 分钟），最小值 60 秒。

为什么是 15 分钟？这是一个经验值：

- 太短（比如 1 分钟）：任务还没执行完，锁就过期了，其他实例会把同一个任务再抢走
- 太长（比如 1 小时）：实例宕机后，锁要等很久才过期，影响故障恢复速度

15 分钟是一个平衡点：正常的刷新流程（下载文件 + 分块 + 上传）通常在几分钟内完成，15 分钟足够了。即使实例宕机，最多 15 分钟后锁自动过期，其他实例可以接管。

配合后面讲的自动心跳续锁机制，长任务也不会因为锁过期而被中断。

4.2 自动心跳续锁机制

获取锁只是第一步。因为一次刷新可能持续很久：下载远程文件、执行分块、上传对象存储，整个流程可能需要几分钟甚至更长。如果锁固定只给 15 分钟，中途就会过期，其他实例会把同一个任务再抢走。

传统的解决方案：手动续锁

在每个耗时操作前，手动调用 renew(lease) 续期：

```
// 下载文件前续锁
lockManager.renew(lease);
downloadFile();

// 执行分块前续锁
lockManager.renew(lease);
chunkDocument();

// 上传文件前续锁
lockManager.renew(lease);
uploadFile();
```

这样做有几个问题：

- 业务代码需要记住在哪些地方续锁，心智负担重
- 如果某个地方忘记续锁，锁就会过期
- 续锁的时机不好把握，太频繁浪费资源，太稀疏又可能过期

自动心跳续锁的思路

既然续锁是一个周期性的动作，为什么不用一个后台线程自动做？

1. 获取锁后，启动一个后台心跳线程
2. 心跳线程周期性调用 `renew(lease)`
3. 业务代码不需要手动续锁，只需在关键阶段检测锁是否失效
4. 任务执行完后，关闭心跳线程

这就是自动心跳续锁机制的核心思路。

心跳间隔怎么计算？ 心跳间隔 = 锁持有时间 / 3，但限制在 5~60 秒之间。

默认配置下：

锁持有时间 = 900 秒（15 分钟）

心跳间隔 = $900 / 3 = 300$ 秒 = 5 分钟

但因为有最大值限制，实际心跳间隔是 60 秒

为什么是 1/3？这是一个经验值：

太频繁（比如 1/10）：心跳请求太多，浪费数据库资源

太稀疏（比如 1/2）：如果某次心跳失败，可能来不及重试就过期了

1/3 是一个平衡点：既不会太频繁，又有足够的容错空间

这个比例也是 Redisson 看门狗机制的默认值，经过大量实践验证。

心跳失败怎么处理？

心跳线程周期性调用 `renew(lease)`，如果续锁失败（返回 `false`），说明锁已经被其他实例抢走了。

这时心跳线程会调用 `heartbeat.markLost()` 标记锁已失效。业务逻辑在关键阶段检测 `heartbeat.isLost()`，如果发现锁已失效，立即中止执行。

为什么不直接抛异常？因为业务逻辑可能正在执行某个操作（比如上传文件），需要等到一个安全的时机才能中止。所以心跳线程只负责标记，业务逻辑自己决定什么时候中止。

4.3 锁失效检测

虽然有自动心跳续锁，但还需要在关键阶段主动检测锁是否失效。

为什么需要主动检测？

因为心跳是异步的，可能业务逻辑还在执行，心跳线程已经发现续锁失败了。如果业务逻辑不检测，可能会继续执行很久才发现锁已经失效。

在哪些阶段检测？

在高成本操作之前检测：

抢占文档运行权之前：如果锁已失效，就不应该再抢占文档了

执行分块之前：分块是最耗时的操作，必须确保锁仍然有效

怎么检测？

`shouldAbortForLeaseLoss()` 方法做了两重检测：

```
private boolean shouldAbortForLeaseLoss(ScheduleLockLease lease,
                                         ScheduleLockHeartbeat heartbeat,
                                         String stage) {

    // 第一重检测：心跳是否已标记 lost
    if (heartbeat.isLost()) {
        log.warn("心跳已标记 lost, 中止执行: scheduleId={}, stage={}",
                lease.scheduleId(), stage);
        return true;
    }

    // 第二重检测：主动尝试续锁一次
    boolean renewed = lockManager.renew(lease);
    if (!renewed) {
        heartbeat.markLost("主动续锁失败 at " + stage);
        log.warn("主动续锁失败, 中止执行: scheduleId={}, stage={}",
                lease.scheduleId(), stage);
        return true;
    }

    return false;
}
```

为什么需要两重检测？

- 只检查心跳标记：可能心跳线程还没来得及执行，业务逻辑已经推进到关键阶段了
- 只主动续锁：可能心跳线程已经发现续锁失败，但业务逻辑还没检测到

两重检测结合，确保锁失效能被及时发现。

4.4 释放锁

任务执行完成后，必须释放锁。释放锁的逻辑很简单：把 lockOwner 和 lockUntil 设为 NULL。但有一个关键点：释放锁时也要检查 lockOwner 是否匹配，防止误释放其他实例的锁。

```
UPDATE t_knowledge_document_schedule
SET lockOwner = NULL, lockUntil = NULL
WHERE id = ?
AND lockOwner = ?
```

这条 SQL 的语义是：只有当 lockOwner 匹配当前 token 时，才允许释放锁。

为什么要验证 lockOwner？

假设不验证，可能会出现这种情况：

- 实例 A 获取锁，开始执行任务
- 实例 A 执行很慢，锁过期了
- 实例 B 获取锁，开始执行同一个任务
- 实例 A 执行完了，释放锁
- 实例 A 误把实例 B 的锁释放了
- 实例 C 获取锁，和实例 B 同时执行同一个任务

所以释放锁时必须验证 lockOwner，确保只释放自己持有的锁。

释放锁的时机

释放锁的时机在 finally 块中，确保无论任务成功、失败还是跳过，锁都一定会被释放：

```
finally {
    // 关闭心跳
    if (heartbeat != null) {
        heartbeat.close();
    }

    // 释放锁
    lockManager.release(lease);
}
```

远程文件变更检测

理解了分布式锁，再来看变更检测机制。定时同步的核心目标是有变化才刷新，所以变更检测是整个机制的关键环节。

1. 为什么需要变更检测

先算一笔账。
假设你配置了每小时同步一次，一天就是 24 次。如果每次都无脑下载完整文件并重新分块：

- 下载文件：可能几十 MB，耗时几秒到几十秒
- 重新分块：可能几百个 chunk，耗时几秒到几分钟
- 调用 Embedding API：可能几百次请求，成本几毛到几块钱
- 写入向量数据库：可能几百条记录，耗时几秒

但实际上，这份文档可能一周才更新一次。也就是说，168 次定时触发中，只有 1 次是真正需要刷新的，其他 167 次都是浪费。所以需要在下载和分块之前，先判断文件是否真的变了。如果没变，直接跳过，节省资源和成本。

2. 两级检测策略

`RemoteFileFetcher.fetchIfChanged()` 实现了两级检测策略：先用 HTTP Header 快速判断，再用内容哈希兜底。

2.1 第一级：HEAD 请求快速判断

先发一个 HEAD 请求，获取远程资源的元数据，不下载文件内容。
HTTP 协议定义了两个标准 Header 用于标识资源版本：

ETag (Entity Tag)：

- 资源的唯一标识符，通常是内容的哈希值或版本号
- 格式："`33a64df551425fcc55e4d42a148795d9f25f89d4`"
- 内容变化时，ETag 会变化

Last-Modified：

- 资源的最后修改时间
- 格式：`Wed, 21 Oct 2015 07:28:00 GMT`
- 内容变化时，Last-Modified 会更新

当前实现的检测逻辑是：如果远程文件的 ETag 或 Last-Modified 任意一个和上次存储的值相同，就直接短路判定文件没变。
这一层的优点：

- 不需要下载文件，只发一个 HEAD 请求
- 速度很快，通常几十毫秒
- 文件没变化时，可以直接在这一层短路返回

这一层的局限：

- 有些服务器不支持 HEAD 请求
- 有些服务器不返回 ETag 或 Last-Modified
- 有些服务器返回的 Header 不可靠（比如每次都变）

所以需要第二层兜底。

2.2 第二级：下载并计算内容哈希

如果 HEAD 请求失败，或者 HEAD 返回的信息没有命中“未变化”短路条件，就进入第二层：下载文件并计算 SHA-256 哈希。
这里有一个巧妙的设计：边下载边计算哈希，只遍历一次数据流。

```
// 伪代码示意
try (InputStream input = response.bodyStream();
    OutputStream output = Files.newOutputStream(tempFile);
    DigestInputStream digestInput = new DigestInputStream(input, sha256Digest)) {

    byte[] buffer = new byte[8192];
    while ((bytesRead = digestInput.read(buffer)) != -1) {
        output.write(buffer, 0, bytesRead);
        // 同时计算哈希
    }

    String hash = bytesToHex(sha256Digest.digest());
}
```

下载完成后，比对计算出的哈希和上次存储的 `lastContentHash`。如果相同，说明内容没变，删除临时文件并返回 `SKIPPED`。
这一层的优点：

- 不依赖服务器返回的 Header，完全基于内容判断
- SHA-256 哈希冲突概率极低，可以认为是内容的唯一标识
- 即使 Header 变了但内容没变，也能正确判断

这一层的成本：

- 需要下载完整文件
- 需要计算 SHA-256（CPU 密集）
- 但相比重新分块和调用 Embedding API，成本还是低很多

2.3 两级策略的配合

两级策略的配合逻辑：

- 1. 先尝试 HEAD 请求，获取 ETag 和 Last-Modified
- 2. 如果任意一个和上次相同，返回 SKIPPED（文件没变）
- 3. 如果 HEAD 请求失败，或者 HEAD 返回的信息没有命中短路条件，进入第二级
- 4. 下载文件并计算 SHA-256
- 5. 如果哈希和上次相同，返回 SKIPPED（内容没变）
- 6. 如果哈希不同，返回 CHANGED（文件有变化）

这是典型的"先 cheap check，再 expensive check"策略。

3. RemoteFetchResult 设计

RemoteFetchResult 实现了 AutoCloseable 接口，用于管理临时文件的生命周期。

为什么需要 AutoCloseable?

因为变更检测会下载文件到临时目录，如果文件没变化，需要删除临时文件。如果忘记删除，会导致临时文件泄漏。

用 AutoCloseable 配合 try-with-resources，可以自动管理临时文件的生命周期：

```
try (RemoteFetchResult fetchResult = remoteFileFetcher.fetchIfChanged(...)) {
    if (!fetchResult.changed()) {
        // 文件没变化，跳过
        return;
    }
    // 文件有变化，使用 fetchResult.tempFile() 处理
}
// 离开作用域时，tempFile 自动删除
```

这样设计的好处：

- 临时文件的生命周期由 try-with-resources 自动管理
- 不会因为异常导致临时文件泄漏
- 调用方不需要手动删除临时文件

RemoteFetchResult 包含哪些信息?

- changed(): 文件是否有变化
- tempFile(): 临时文件路径（只有 changed=true 时才有）
- size(): 文件大小
- contentHash(): SHA-256 哈希
- etag(): ETag
- lastModified(): Last-Modified
- message(): 跳过原因（比如"远程文件未变化"、"内容哈希未变化"）

这些信息会被记录到 exec 表和 schedule 表，方便追溯和调试。

小结

这一篇讲的是定时同步机制的基础设施层，核心要点：

- 数据模型先行**：两张表（任务主表 + 执行历史表）支撑整个机制，表结构设计决定了后续的实现方式。
- 职责分离**：五个核心组件各司其职，调度入口、锁管理、刷新编排、状态管理、文档状态管理，每个组件只负责一件事。
- 数据库租约锁**：不新增基础设施，用两个字段（lockOwner + lockUntil）实现分布式锁，配合自动心跳续锁机制，业务代码无感知。
- 两级变更检测**：先用 HTTP Header 快速判断，再用内容哈希兜底，大部分情况下不需要重新分块。

理解了这些基础设施，下一篇就能看懂整个调度系统是怎么跑起来的：调度引擎如何扫描任务、执行流程的三个阶段、异常恢复机制、配置调优和常见问题。

